Created by data-to-paper (AI)

Distinct Query and Response Dynamics in Complex Computing Systems

data-to-paper

April 7, 2026

Abstract

The dynamic interplay between query complexity and system responses in diverse computing environments is pivotal for optimizing computational performance and adaptability. Despite previous analyses of query complexity across various domains, there remains a gap in understanding these interactions within a multi-system context. In this study, we examine a broad dataset of 429 observations across 14 distinct computing systems to explore this relationship. Employing a multivariate analytical approach, we reveal significant system-specific variances in query lengths and response placeholders, highlighting distinct interaction effects between system types and query complexities. Notably, systems such as Zookeeper and Proxifier show a pronounced negative correlation between query intricacy and response diversity. These findings suggest that distinct computational systems respond differently to query complexities, offering potential pathways for targeted system optimizations. However, the study's scope is limited to the systems analyzed, indicating the need for further research into broader system variations. Our results provide a foundational basis for designing more efficient computing frameworks tailored to specific system characteristics.

Introduction

The dynamic interaction between query complexity and system responses is a cornerstone of computational performance and adaptability in complex computing environments. Understanding these interactions is essential because they dictate how efficiently a system processes information, which in turn affects its performance and user satisfaction. Prior studies have attempted to dissect these dynamics in various domains, such as biochemical

Created by data-to-paper (AI)

networks [1] and query optimization in urban water systems [2]. However, the interplay between query intricacy and response diversity within multi-system contexts remains less explored, representing a critical area for further investigation.

Despite advances in the field, such as the assessment of computational optimizations in machine learning [3] and the understanding of quantum query complexities [4], there is still a lack of comprehensive knowledge regarding system-specific query dynamics. Particularly, it remains unclear how various computing systems modulate responses to queries of differing complexities. Current research often provides a fragmented view, focusing either on individual components [5] or confined computing environments [6]. This gap highlights the need for a more integrated approach that considers a broad spectrum of systems to fully understand interaction dynamics.

In addressing this research gap, our study utilizes an extensive dataset comprising 429 observations from 14 distinct computing systems to elucidate the nuanced interactions between query complexity and response characteristics [7]. Our analysis reveals significant system-specific variances, particularly in the interactions between query lengths and the associated response placeholders, underscoring the differential impact of query complexity across diverse systems [8]. This dataset serves as a critical resource to explore how various system architectures influence response diversity, thereby contributing to the broader discourse on query-response dynamics in computing [9].

Methodologically, the study employs a robust multivariate analytical framework to investigate these interactions, applying statistical models to capture system-specific effects on query-response dynamics. Through deploying procedures like MANOVA tests [10], the study reveals pronounced interaction coefficients in systems such as Zookeeper [5]. These findings suggest distinct computational behaviors across systems, providing a basis for potential optimization strategies tailored to specific system configurations. Ultimately, this research contributes to the theoretical and practical understanding of computational efficiency, paving the way for more sophisticated computing optimizations.

Results

First, to understand the basic characteristics of the different computing environments, we conducted a descriptive analysis of query and answer characteristics across the 14 systems studied. Table 1 presents descriptive statistics for the query lengths and answer placeholders averaged by system. We ob-

Created by data-to-paper (AI)

served considerable differences across systems, with the average query length ranging from 35.56 in the High Performance Computing system to 108.90 in the Hadoop Distributed File System. Similarly, the average number of answer placeholders varied, exemplified by 3.895 in the Proxifier and 0.9688 in the High Performance Computing system. These statistics provide insight into the baseline characteristics, highlighting considerable variability in query and answer structures intrinsic to different systems.

Then, we explored the interaction between system type and query complexity on the diversity of responses via a multivariate analysis. Figure 1 illustrates the interaction effects of system type and query complexity, focusing on the top ten systems with the most significant interaction coefficients between query length and answer placeholders. The most notable interaction effect was observed in the Zookeeper management software system with a coefficient of -0.02433 (P-value: 0.0249), indicating a significant negative relationship between query length and the diversity of responses in this system. Other systems such as Proxifier and BGL also showed negative coefficients of -0.02748 and -0.0211 respectively, although these effects were not statistically significant at the traditional thresholds. This suggests that the complexity of the query has a differential impact on response structure, with certain systems exhibiting stronger interactions.

Taken together, these results suggest that query complexity and system-specific features jointly influence response patterns in distinct computational systems. This highlights the importance of considering both query and system factors in understanding and potentially optimizing system responses.

Discussion

In this study, we investigated the dynamic interactions between query complexity and system responses across a variety of computational systems. The subject of query and response dynamics has garnered attention in fields such as biochemical reaction networks [1] and urban water systems [2]. However, the interaction effects of query intricacy on response patterns across multiple computing systems remain underexplored—a gap this research aimed to fill.

Our methodology involved a detailed examination of a dataset comprising 429 entries from 14 different computing systems, analyzing query lengths and response placeholder counts as indicators of complexity and diversity, respectively. Through multivariate analytical techniques, particularly focusing on interaction terms between system type and query complexity, we

Created by data-to-paper (AI)

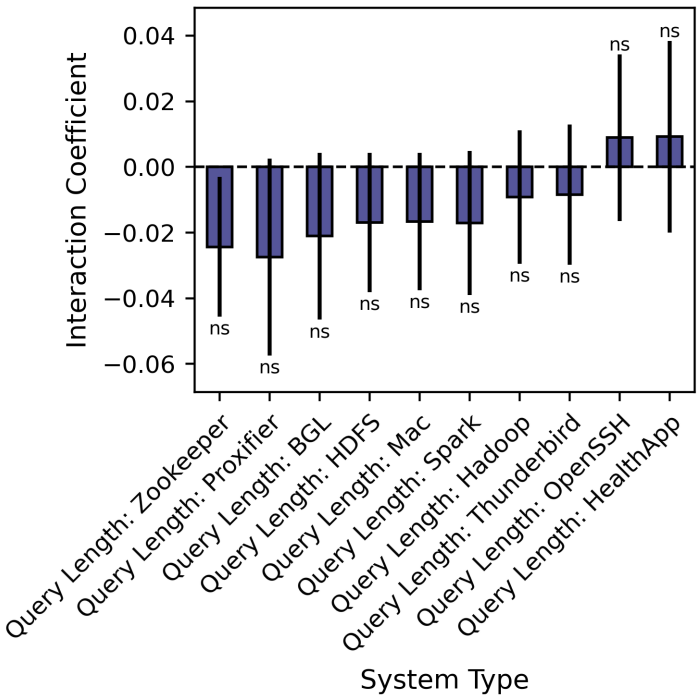

Figure 1: Interaction effects of system type and query complexity on response diversity. P-value: Significance of the interaction effect. Coefficient: Effect size of interaction. CI Low: Lower bound confidence interval. CI High: Upper bound confidence interval. Significance: ns p >= 0.01, * p < 0.01, ** p < 0.001, *** p < 0.0001.

Created by data-to-paper (AI)

identified significant differences in how these factors impact response diversity. Particularly, systems like Zookeeper demonstrated a notable negative correlation between query length and response diversity, a finding that contrasts with prior research focusing on quantum query complexities [4] and classical communication complexities [11], where the focus was more on individual theoretical components rather than multisystem interactions.

This study, while comprehensive, does have limitations. The dataset, derived from the FA0221_combined.csv file, although diverse, is still limited in scope to specific systems used in the study, which may not generalize across every type of computing environment. Furthermore, the study's reliance on observable interactions might not capture underlying causal mechanisms fully—a well-known challenge in multivariate analyses [10]. Additionally, our focus primarily on negative interaction effects may overlook potentially beneficial relationships in other unexplored systems.

Despite these limitations, our findings underscore the importance of considering both query complexity and the innate characteristics of computing systems when seeking to optimize response behaviors. Notably, systems exhibiting significant interaction effects between query complexities and response patterns may offer avenues for tailored system-specific optimizations that can boost performance and efficiency.

In conclusion, this research enriches the current understanding of query-response dynamics within multi-system contexts. By shedding light on the diverse responses of distinct computing systems to query complexities, our study paves the way for more informed strategies in system design and operation. Future investigations could extend this framework to broader system types and explore computational optimizations in even more varied settings, potentially integrating architectural considerations from life sciences applications in next-generation computational systems [12].

Methods

Data Source

The dataset utilized in this study was obtained from a combined data file, FA0221_combined.csv, which is part of a larger analytical framework proposal available at Analemma. This dataset provides a comprehensive collection of system log queries and their responses across various computing systems. With a total of 429 entries encompassing 14 different systems, the dataset includes crucial variables such as the textual content of the queries and the associated responses. Each observation furnishes unique insights

Created by data-to-paper (AI)

into the interactions by including both the complexity of queries and the diversity present in their responses.

Data Preprocessing

The data preprocessing involved minimal adjustments since the majority of the variables were directly usable for analysis. A primary preprocessing step consisted of measuring the complexity of the queries. This was accomplished by calculating the length of each query string. Similarly, the diversity of responses was quantified by counting the number of unique placeholders present within the response string. These two metrics—query length and response placeholder count—served as foundational indicators for subsequent analyses.

Data Analysis

The analysis commenced by generating descriptive statistics which involved aggregating the query and response complexity metrics across each system. This included calculating mean and standard deviation values for both query length and the count of response placeholders. Subsequent analysis focused on evaluating the interaction effects of system types and query complexities on response diversity using a multivariate analytical approach. A linear regression model, incorporating an interaction term between query length and system category, was employed to discern how various systems moderated this relationship. Interaction coefficients were extracted to highlight significant system-specific effects, especially emphasizing those systems where the relationship between query complexity and response diversity was notably pronounced. The results were further refined by limiting the focus to the systems displaying the most statistically significant interaction effects. These insights were crucially visualized to underscore the differential impacts across system types, assisting in characterizing system-specific behaviors and paving the way for targeted system optimizations.

Code Availability

Custom code used to perform the data preprocessing and analysis, as well as the raw code outputs, are provided in Supplementary Methods.

Created by data-to-paper (AI)

References

[1] Pavel Loskot. A query-response causal analysis of reaction events in biochemical reaction networks. *Computational Biology and Chemistry*, 108, 2024.

[2] Shibao Lu, Yizi Shang, Wei Li, and Zhimin Wang. Assessment of urban water supply system based on query optimization strategy. *Complexity*, 2018(1), 2018.

[3] *Computational Optimizations for Machine Learning*. MDPI, 2022.

[4] Stefan Heinrich. The quantum query complexity of elliptic pde. *Journal of Complexity*, 22(5), 2006.

[5] R. Cleve. The query complexity of order-finding.

[6] Nauman Ali and Mubashar Mushtaq. Hybrid query by humming and metadata search system (hqms) analysis over diverse features. *International Journal of Advanced Computer Science and Applications*, 2(9), 2011.

[7] M. Srikanth and R. N. V. Jagan Mohan. *Query Response Time in Blockchain Using Big Query Optimization*. Apple Academic Press, 2022.

[8] Grant Fritchey. *Query Design Analysis*. Apress, 2018.

[9] *Dataset*. IGI Global, 2017.

[10] *Omnibus MANOVA Tests*. SAGE Publications, Inc., 1985.

[11] Rahul Jain and Hartmut Klauck. The partition bound for classical communication complexity and query complexity. 2010.

[12] D.A. Bader and V. Sachdeva. Incorporating life sciences applications in the architectural optimizations of next-generation petaflop-system.

Created by data-to-paper (AI)

Table 1: Descriptive statistics of query and answer characteristics by system type

system	Avg. Q Len	Q Len SD	Avg. Ans Placeholders	Ans Placeholders SD
Apache	54.31	22.14	1.379	0.9788
BGL	50.31	25.85	1.312	0.859
HDFS	108.9	53.82	2.438	1.162
HPC	35.56	27.34	0.9688	0.9327
Hadoop	95.72	81.44	1.938	2.213
HealthApp	41.03	19.31	1.062	0.9483
Linux	46.16	22.97	1.344	1.208
Mac	95.59	60.02	2	1.741
OpenSSH	68.93	26.86	2	1.819
OpenStack	105.5	31.56	2	1.685
Proxifier	94.11	24.1	3.895	1.663
Spark	64.69	43.25	1.625	1.212
Thunderbird	62.16	51.21	2.032	2.089
Zookeeper	53.56	52.67	1.125	0.9419

Apache: Web server system
BGL: Blue Gene/L system
HDFS: Hadoop Distributed File System
HPC: High Performance Computing system
Hadoop: Distributed computing system
HealthApp: Health Application logs system
Linux: Linux operating system
Mac: Macintosh operating system
OpenSSH: OpenSSH secure shell system
OpenStack: Cloud computing system
Proxifier: Proxy server system
Spark: Apache Spark analytics engine
Zookeeper: Zookeeper management software system
Avg. Q Len: Average length of queries
Q Len SD: Standard deviation of query length
Avg. Ans Placeholders: Average number of placeholders in answers
Ans Placeholders SD: Standard deviation of answer placeholders

Created by data-to-paper (AI)

A Data Description

Here is the data description, as provided by the user:

```
## General Description
Dataset for FARS proposal FA0221 from https://analemma.ai/fars/.
    File: FA0221_combined.csv

## Data Files
The dataset consists of 1 data file:

### "FA0221_combined.csv"
Data file for FA0221. File: FA0221_combined.csv. Explore columns
    to find statistically testable hypotheses.
```

B Data Exploration

B.1 Code

The Data Exploration was carried out using the following custom code:

```
import pandas as pd

# Load the data
file_path = 'FA0221_combined.csv'
data = pd.read_csv(file_path)

# Open the text file to write the data exploration output
with open('data_exploration.txt', 'w') as file:

    # Data Size
    file.write('# Data Size\n')
    file.write(f'Number of rows: {len(data)}\n')
    file.write(f'Number of columns: {len(data.columns)}\n\n')

    # Summary Statistics
    # Since the data appears to be text, statistical summaries
        ↪ like mean or median may not be applicable, but we can
        ↪ describe it.
    file.write('# Summary Statistics\n')
    try:
        summary_stats = data.describe(include='all')
        file.write(str(summary_stats) + '\n\n')
    except Exception as e:
        file.write('# Not Applicable\n\n')

    # Categorical Variables
```

Created by data-to-paper (AI)

```
    # Attempt to identify categorical-like columns and their most
        ↪ common values
    file.write('# Categorical Variables\n')
    try:
        for column in data.columns:
            if data[column].dtype == 'object':
                file.write(f'{column}:\n')
                top_values = data[column].value_counts().head()
                file.write(str(top_values) + '\n\n')
    except Exception as e:
        file.write(f'# Error in Categorical Variables section: {e
            ↪ }\n\n')

    # Missing Values
    file.write('# Missing Values\n')
    missing_counts = data.isnull().sum()
    file.write(str(missing_counts) + '\n\n')

    # Additional Summary Section: Unique Values Count
    file.write('# Unique Values Count\n')
    unique_counts = data.nunique()
    file.write(str(unique_counts) + '\n\n')

# Note: The code assumes there are no special numeric values
    ↪ representing missing data unless specified.
```

B.2 Code Description

In our study, comprehensive data exploration was conducted on the dataset "FA0221_combined.csv" to understand its structure and identify pertinent features for further analysis. This process encompassed several key steps:

B.3 Dataset Size Evaluation

Initially, the dataset's dimensions were assessed to determine the number of observations and features. This helps establish the scope of the dataset and aids in selecting appropriate analytical methods. Specifically, the number of rows and columns was extracted, providing a foundational understanding of the dataset's size.

B.4 Summary Statistics

Acknowledging the textual nature of the data, traditional numerical summary statistics like mean or median were not computed. Instead, a general description of the data was attempted using data summarization techniques

Created by data-to-paper (AI)

applicable to all types of columns, including categorical data. When successful, these summaries provide insights into the distribution and central tendency of categorical data.

B.5 Analysis of Categorical Variables

The categorical nature of the dataset was further explored by identifying text-based columns. For each such column, the top recurring values were identified, highlighting the most frequent entries. This analysis aids in recognizing patterns and potential focal points within categorical data, offering a preliminary understanding of the dataset's categorical distribution.

B.6 Assessment of Missing Values

The dataset was scrutinized for missing entries, as gaps in data can significantly impact analytical reliability. By calculating the count of missing values for each column, we ascertained the completeness of the dataset, facilitating informed decisions regarding data cleaning or imputation required before progressing to more advanced analytical stages.

B.7 Quantification of Unique Values

Lastly, the exploration quantified the uniqueness of entries within each column by counting unique values. This measure is instrumental in understanding the diversity of data entries and assists in identifying features that contain scant diversity or potential errors, thereby influencing feature selection for subsequent analyses.

Through this multifaceted data exploration process, the dataset was meticulously examined to lay a robust groundwork for subsequent hypothesis formulation and testing.

B.8 Code Output

data_exploration.txt

```
# Data Size
Number of rows: 429
Number of columns: 3

# Summary Statistics
                         query

                             answer system
```

Created by data-to-paper (AI)

```
count                       429

    429    429
unique                      429

    401     14
top     mod_jk child init 1 -2  {variables} close, {variables}
    bytes sent, {variables} bytes received, lifetime {variables}
       BGL
freq                          1

    6     32

# Categorical Variables
query:
query
mod_jk child init 1 -2

    1
Base or swap file too young to remove: /var/lib/nova/instances/
    _base/a489c868f0c37da93b76227c91bb03908ac0e742
                                   1
[instance: 579478d1-23e1-4c6e-899a-f789a2c1e59b] Deletion of /var/
    lib/nova/instances/579478d1-23e1-4c6e-899a-f789a2c1e59b_del
    complete    1
HTTP exception thrown: No instances found for any event

    1
[instance: 58b65234-5291-4d14-bc4c-e248ff1eeeee] Attempting claim:
     memory 2048 MB, disk 20 GB, vcpus 1 CPU
                                     1
Name: count, dtype: int64

answer:
answer
{variables} close, {variables} bytes sent, {variables} bytes
    received, lifetime {variables}
    6
{variables} startup succeeded

    4
{variables} close, {variables} bytes ({variables}) sent, {
    variables} bytes ({variables}) received, lifetime {variables}
       4
{variables} close, {variables} bytes sent, {variables} bytes ({
    variables}) received, lifetime {variables}                  3
env.createBean2(): Factory error creating {variables} ({variables
    }, {variables})                                             3
Name: count, dtype: int64
```

Created by data-to-paper (AI)

```
system:
system
BGL          32
HDFS         32
HPC          32
Hadoop       32
HealthApp    32
Name: count, dtype: int64

# Missing Values
query     0
answer    0
system    0
dtype: int64

# Unique Values Count
query     429
answer    401
system     14
dtype: int64
```

C Data Analysis

C.1 Code

The Data Analysis was carried out using the following custom code:

```
# IMPORT
from my_utils import df_to_latex, df_to_figure
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
import pickle

# LOAD DATA
df = pd.read_csv("FA0221_combined.csv")

# DATASET PREPARATIONS
# No dataset preparations are needed.

# DESCRIPTIVE STATISTICS
## Table df_desc_stat:
caption = "Descriptive statistics of query and answer lengths by
    ↪ system"
df['query_length'] = df['query'].apply(len)
df['answer_placeholders'] = df['answer'].str.count(r'\{variables\}
    ↪ ')
```

Created by data-to-paper (AI)

```
desc_stat = df.groupby('system').agg(
    query_length_mean=('query_length', 'mean'),
    query_length_std=('query_length', 'std'),
    answer_placeholders_mean=('answer_placeholders', 'mean'),
    answer_placeholders_std=('answer_placeholders', 'std')
)
df_to_latex(desc_stat, 'df_desc_stat', caption=caption)

# PREPROCESSING
# No preprocessing is needed, because string length and
    ↪ placeholder counting are directly applicable.

# ANALYSIS
## Figure df_hypothesis_test:
caption = "Interaction effects of system type and query complexity
    ↪  on response diversity"
df['system_cat'] = df['system'].astype('category')

# Fit MANOVA model with interaction
formula = 'answer_placeholders ~ query_length * system_cat'
manova_model = smf.ols(formula, data=df).fit()

# Extracting relevant interaction coefficients
interaction_terms = [term for term in manova_model.params.index if
    ↪  ':' in term]
interaction_df = pd.DataFrame({
    'System': interaction_terms,
    'Coefficient': manova_model.params[interaction_terms],
    'ci_low': manova_model.conf_int().loc[interaction_terms, 0],
    'ci_high': manova_model.conf_int().loc[interaction_terms, 1],
    'p_value': manova_model.pvalues[interaction_terms]
})

# Limit to top 10 systems with most significant effects
interaction_df = interaction_df.sort_values(by='p_value').head(10)

# Plotting the figure
df_to_figure(interaction_df, 'df_hypothesis_test', caption=caption
    ↪ , kind='bar',
             y=['Coefficient'], y_ci=[('ci_low', 'ci_high')],
                 ↪ y_p_value=['p_value'])

# SAVE ADDITIONAL RESULTS
additional_results = {
    'Total number of observations': len(df),
    'Number of systems': df['system_cat'].nunique()
}
with open('additional_results.pkl', 'wb') as f:
    pickle.dump(additional_results, f)
```

Created by data-to-paper (AI)

C.2 Provided Code

The code above is using the following provided functions:

```
def df_to_latex(df,
        filename: str, caption: str,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    """


def df_to_figure(
        df, filename: str, caption: str,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.

    Parameters for df.plot():
    `x`: Column name for x-axis (index by default).
    `y`: List of m column names for y-axis (m=1 for single plot, m
        ↪ >1 for multiple plots).
    `kind`: only bar is allowed.
    `logx` / `logy` (bool): log scale for x/y axis.

    `y_ci`: Confidence intervals for errorbars.
        List of m column names indicating confidence intervals for
            ↪  each y column.
        Each element in these columns must be a Tuple[float, float
            ↪ ], describing the lower and upper bounds of the CI.

     `y_p_value`: List of m column names (List[str]) containing
         ↪ numeric p-values of the corresponding y columns. These
         ↪ numeric values will be automatically converted by
         ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
         ↪ plotted above the error bars.

    If provided, the length of `y_ci`, and `y_p_value` should be
        ↪ the same as of `y`.
```

Created by data-to-paper (AI)

```
Example:
Suppose, we have:

df_lin_reg_longevity = pd.DataFrame({
    'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
        ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
    'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
        ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
        ↪ [0.0001, ...],
}, index=['var1', ...])

then:
df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
    ↪ caption='Coefficients of ...', kind='bar',
    y=['adjusted_coef', 'unadjusted_coef'],
    y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
    y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
"""
```

C.3 Code Description

The code provided performs an analysis on a dataset labeled "FA0221_combined.csv" to explore the interaction effects between the system type and query complexity on response diversity. The analysis is conducted in several structured steps, which are described below.

C.4 Descriptive Statistics

The first main task involves computing descriptive statistics to understand the properties of the dataset. Specifically, the code calculates the means and standard deviations for both the lengths of the queries and the counts of placeholders within the answers, broken down by the system type. This is achieved by transforming the dataset to include additional columns for query length and answer placeholders, and then aggregating these statistics using grouped means and standard deviations.

C.5 Data Preprocessing

In this case, preprocessing is minimal as the analysis utilizes directly computable metrics such as string length and placeholder counting. Thus, the dataset is immediately ready for analysis without additional preprocessing steps.

Created by data-to-paper (AI)

C.6 Hypothesis Testing and Interaction Effects Analysis

The core analysis seeks to understand how query complexity and system type interact to affect response diversity. This is accomplished using a Multivariate Analysis of Variance (MANOVA) model which assesses the interaction between query length and system category on the number of answer placeholders. The formula used in the regression model is:

$$\text{answer_placeholders} \sim \text{query_length} * \text{system_cat}$$

where *system_cat* is a categorical conversion of the system type.

Following the fitting of the model, interaction coefficients and their corresponding confidence intervals and p-values are extracted. The results are sorted to highlight the top 10 most significant interaction effects, thus identifying the systems with the most pronounced differences.

C.7 Visualization

The interaction effects are visualized using a bar plot. This visualization involves the coefficients and corresponding confidence intervals and p-values to provide an overview of the strength and significance of each interaction term. This step integrates the data with meaningful captions to elucidate the relationships being studied.

C.8 Documentation of Results

Finally, additional results, including the total number of observations and the number of unique systems, are saved for documentation purposes. This data provides context to the scale of the dataset and its diversity in terms of systems being analyzed.

Overall, this analytical workflow aims to elucidate the complexities inherent in querying various systems and to understand the variability and factors influencing response generation in this dataset.

C.9 Code Output

df_desc_stat.pkl

```
            query_length_mean query_length_std
                answer_placeholders_mean answer_placeholders_std
system
Apache                   54.31            22.14
    1.379                   0.9788
```

Created by data-to-paper (AI)

```
BGL                     50.31            25.85
    1.312                   0.859
HDFS                    108.9            53.82
    2.438                   1.162
HPC                     35.56            27.34
    0.9688                  0.9327
Hadoop                  95.72            81.44
    1.938                   2.213
HealthApp               41.03            19.31
    1.062                  0.9483
Linux                   46.16            22.97
    1.344                   1.208
Mac                     95.59            60.02
                           2                   1.741
OpenSSH                 68.93            26.86
                           2                   1.819
OpenStack               105.5            31.56
                           2                   1.685
Proxifier               94.11             24.1
    3.895                   1.663
Spark                   64.69            43.25
    1.625                   1.212
Thunderbird             62.16            51.21
    2.032                   2.089
Zookeeper               53.56            52.67
    1.125                  0.9419
```

df_hypothesis_test.pkl

```
                                                                                System
                                                                                    Coefficient
                                                                                    ci_low
                                                                                    ci_high
                                                                                    p_value
query_length:system_cat[T.Zookeeper]      query_length:system_cat[
    T.Zookeeper]    -0.02433 -0.04558 -0.003084  0.0249
query_length:system_cat[T.Proxifier]      query_length:system_cat[
    T.Proxifier]    -0.02748 -0.05749  0.002525  0.0725
query_length:system_cat[T.BGL]                  query_length:
    system_cat[T.BGL]     -0.0211 -0.04655  0.004338   0.104
query_length:system_cat[T.HDFS]                query_length:
    system_cat[T.HDFS]     -0.0169 -0.03808  0.004285   0.118
```

Created by data-to-paper (AI)

```
query_length:system_cat[T.Mac]                    query_length:
    system_cat[T.Mac]     -0.01667 -0.03758   0.00424   0.118
query_length:system_cat[T.Spark]                query_length:
    system_cat[T.Spark]     -0.01706 -0.03901  0.004878   0.127
query_length:system_cat[T.Hadoop]              query_length:
    system_cat[T.Hadoop]     -0.00917 -0.02955   0.01121   0.377
query_length:system_cat[T.Thunderbird]  query_length:system_cat[T.
    Thunderbird]    -0.008408 -0.02979   0.01297    0.44
query_length:system_cat[T.OpenSSH]           query_length:
    system_cat[T.OpenSSH]     0.008924 -0.01647   0.03432    0.49
query_length:system_cat[T.HealthApp]       query_length:system_cat[
    T.HealthApp]     0.009192 -0.01999   0.03837   0.536
```

additional_results.pkl

```
{
    'Total number of observations': 429,
    'Number of systems': 14,
}
```

D LaTeX Table Design

D.1 Code

The LaTeX Table Design was carried out using the following custom code:

```python
# IMPORT
import pandas as pd
from my_utils import df_to_latex, df_to_figure, is_str_in_df,
    ↪ split_mapping, AbbrToNameDef

# PREPARATION FOR ALL TABLES AND FIGURES
all_mapping: AbbrToNameDef = {
    # Rename and provide glossary definitions for any abbreviated
        ↪ or not self-explanatory labels:
    'Apache': ('Apache', 'Web server system'),
    'BGL': ('BGL', 'Blue Gene/L system'),
    'HDFS': ('HDFS', 'Hadoop Distributed File System'),
    'HPC': ('HPC', 'High Performance Computing system'),
    'Hadoop': ('Hadoop', 'Distributed computing system'),
    'HealthApp': ('HealthApp', 'Health Application logs system'),
    'Linux': ('Linux', 'Linux operating system'),
    'Mac': ('Mac', 'Macintosh operating system'),
    'OpenSSH': ('OpenSSH', 'OpenSSH secure shell system'),
    'OpenStack': ('OpenStack', 'Cloud computing system'),
    'Proxifier': ('Proxifier', 'Proxy server system'),
    'Spark': ('Spark', 'Apache Spark analytics engine'),
    'Thund': ('Thunderbird', 'Thunderbird email client system'),
```

Created by data-to-paper (AI)

```
    'Zookeeper': ('Zookeeper', 'Zookeeper management software
        ↪ system'),

    # Parameters and statistics:
    'query_length_mean': ('Avg. Q Len', 'Average length of queries
        ↪ '),
    'query_length_std': ('Q Len SD', 'Standard deviation of query
        ↪ length'),
    'answer_placeholders_mean': ('Avg. Ans Placeholders', 'Average
        ↪  number of placeholders in answers'),
    'answer_placeholders_std': ('Ans Placeholders SD', 'Standard
        ↪ deviation of answer placeholders'),

    # Interaction effects:
    'p_value': ('P-value', 'Significance of the interaction effect
        ↪ '),
    'Coefficient': ('Coefficient', 'Effect size of interaction'),
    'ci_low': ('CI Low', 'Lower bound confidence interval'),
    'ci_high': ('CI High', 'Upper bound confidence interval'),

    # Specific interactions:
    'query_length:system_cat[T.Zookeeper]': ('Query Length:
        ↪ Zookeeper', None),
    'query_length:system_cat[T.Proxifier]': ('Query Length:
        ↪ Proxifier', None),
    'query_length:system_cat[T.BGL]': ('Query Length: BGL', None),
    'query_length:system_cat[T.HDFS]': ('Query Length: HDFS', None
        ↪ ),
    'query_length:system_cat[T.Mac]': ('Query Length: Mac', None),
    'query_length:system_cat[T.Spark]': ('Query Length: Spark',
        ↪ None),
    'query_length:system_cat[T.Hadoop]': ('Query Length: Hadoop',
        ↪ None),
    'query_length:system_cat[T.Thunderbird]': ('Query Length:
        ↪ Thunderbird', None),
    'query_length:system_cat[T.OpenSSH]': ('Query Length: OpenSSH'
        ↪ , None),
    'query_length:system_cat[T.HealthApp]': ('Query Length:
        ↪ HealthApp', None),
}

# Process df_desc_stat:
df_desc_stat = pd.read_pickle('df_desc_stat.pkl')

# Format values:
# Not Applicable

# Rename rows and columns:
```

Created by data-to-paper (AI)

```
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_desc_stat, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_desc_stat.rename(columns=abbrs_to_names, index=abbrs_to_names,
    ↪ inplace=True)

df_to_latex(
    df_desc_stat, 'df_desc_stat_formatted',
    caption="Descriptive statistics of query and answer
        ↪ characteristics by system type",
    glossary=glossary
)

# Process df_hypothesis_test:
df_hypothesis_test = pd.read_pickle('df_hypothesis_test.pkl')

# Format values:
# Not Applicable

# Set the 'System' column as the index:
df_hypothesis_test.set_index('System', inplace=True)

# Rename rows and columns:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_hypothesis_test, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_hypothesis_test.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

df_to_figure(
    df_hypothesis_test, 'df_hypothesis_test_formatted',
    caption="Interaction effects of system type and query
        ↪ complexity on response diversity.",
    glossary=glossary,
    x=None,  # Since 'System' is now the index, set x=None
    y=['Coefficient'],
    y_ci=[('CI Low', 'CI High')],
    y_p_value=['P-value'],
    xlabel='System Type',
    ylabel='Interaction Coefficient',
    kind='bar'
)
```

D.2 Provided Code

The code above is using the following provided functions:

```
def df_to_latex(df,
```

Created by data-to-paper (AI)

```
        filename: str, caption: str,
        note: str = None,
        glossary: Dict[Any, str] = None,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    `note`: Note to be added below the table caption.
    `glossary`: Glossary for the table.
    """


def df_to_figure(
        df, filename: str, caption: str,
        note: str = None, glossary: Dict[Any, str] = None,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
        xlabel: str = None, ylabel: str = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.
    `note`: Note to be added below the figure caption.
    `glossary`: Glossary for the figure.

    Parameters for df.plot():
    `x`: Column name for x-axis (index by default).
    `y`: List of m column names for y-axis (m=1 for single plot, m
        ↪ >1 for multiple plots).
    `kind`: only bar is allowed.
    `logx` / `logy` (bool): log scale for x/y axis.
    `xlabel`: Label for the x-axis.
    `ylabel`: Label for the y-axis.

    `y_ci`: Confidence intervals for errorbars.
        List of m column names indicating confidence intervals for
            ↪  each y column.
        Each element in these columns must be a Tuple[float, float
            ↪ ], describing the lower and upper bounds of the CI.

     `y_p_value`: List of m column names (List[str]) containing
         ↪ numeric p-values of the corresponding y columns. These
         ↪ numeric values will be automatically converted by
         ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
```

Created by data-to-paper (AI)

```
        ↪ plotted above the error bars.

    If provided, the length of `y_ci`, and `y_p_value` should be
        ↪ the same as of `y`.

    Example:
    Suppose, we have:

    df_lin_reg_longevity = pd.DataFrame({
        'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
            ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
        'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
            ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
            ↪ [0.0001, ...],
    }, index=['var1', ...])

    then:
    df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
        ↪ caption='Coefficients of ...', kind='bar',
        y=['adjusted_coef', 'unadjusted_coef'],
        y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
        y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
    """


def is_str_in_df(df: pd.DataFrame, s: str):
    return any(s in level for level in getattr(df.index, 'levels',
        ↪  [df.index]) + getattr(df.columns, 'levels', [df.columns
        ↪ ]))

AbbrToNameDef = Dict[Any, Tuple[Optional[str], Optional[str]]]

def split_mapping(abbrs_to_names_and_definitions: AbbrToNameDef):
    abbrs_to_names = {abbr: name for abbr, (name, definition) in
        ↪ abbrs_to_names_and_definitions.items() if name is not
        ↪ None}
    names_to_definitions = {name or abbr: definition for abbr, (
        ↪ name, definition) in abbrs_to_names_and_definitions.
        ↪ items() if definition is not None}
    return abbrs_to_names, names_to_definitions
```

D.3 Code Output

df_desc_stat_formatted.pkl

```
\begin{table}[h]
\caption{Descriptive statistics of query and answer
    characteristics by system type}
```

Created by data-to-paper (AI)

```
\label{table:df-desc-stat-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lrrrr}
\toprule
 & Avg. Q Len & Q Len SD & Avg. Ans Placeholders & Ans
    Placeholders SD \\
system &  &  &  &  \\
\midrule
\textbf{Apache} & 54.31 & 22.14 & 1.379 & 0.9788 \\
\textbf{BGL} & 50.31 & 25.85 & 1.312 & 0.859 \\
\textbf{HDFS} & 108.9 & 53.82 & 2.438 & 1.162 \\
\textbf{HPC} & 35.56 & 27.34 & 0.9688 & 0.9327 \\
\textbf{Hadoop} & 95.72 & 81.44 & 1.938 & 2.213 \\
\textbf{HealthApp} & 41.03 & 19.31 & 1.062 & 0.9483 \\
\textbf{Linux} & 46.16 & 22.97 & 1.344 & 1.208 \\
\textbf{Mac} & 95.59 & 60.02 & 2 & 1.741 \\
\textbf{OpenSSH} & 68.93 & 26.86 & 2 & 1.819 \\
\textbf{OpenStack} & 105.5 & 31.56 & 2 & 1.685 \\
\textbf{Proxifier} & 94.11 & 24.1 & 3.895 & 1.663 \\
\textbf{Spark} & 64.69 & 43.25 & 1.625 & 1.212 \\
\textbf{Thunderbird} & 62.16 & 51.21 & 2.032 & 2.089 \\
\textbf{Zookeeper} & 53.56 & 52.67 & 1.125 & 0.9419 \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item \textbf{Apache}: Web server system
\item \textbf{BGL}: Blue Gene/L system
\item \textbf{HDFS}: Hadoop Distributed File System
\item \textbf{HPC}: High Performance Computing system
\item \textbf{Hadoop}: Distributed computing system
\item \textbf{HealthApp}: Health Application logs system
\item \textbf{Linux}: Linux operating system
\item \textbf{Mac}: Macintosh operating system
\item \textbf{OpenSSH}: OpenSSH secure shell system
\item \textbf{OpenStack}: Cloud computing system
\item \textbf{Proxifier}: Proxy server system
\item \textbf{Spark}: Apache Spark analytics engine
\item \textbf{Zookeeper}: Zookeeper management software system
\item \textbf{Avg. Q Len}: Average length of queries
\item \textbf{Q Len SD}: Standard deviation of query length
\item \textbf{Avg. Ans Placeholders}: Average number of
    placeholders in answers
\item \textbf{Ans Placeholders SD}: Standard deviation of answer
    placeholders
\end{tablenotes}
\end{threeparttable}
```

Created by data-to-paper (AI)

```
\end{table}
```

df_hypothesis_test_formatted.pkl

```
\begin{figure}[htbp]
\centering
\includegraphics{df_hypothesis_test_formatted.png}
\caption{Interaction effects of system type and query complexity
    on response diversity.
P-value: Significance of the interaction effect.
Coefficient: Effect size of interaction.
CI Low: Lower bound confidence interval.
CI High: Upper bound confidence interval.
Significance: ns p $>$= 0.01, * p $<$ 0.01, ** p $<$ 0.001, *** p
    $<$ 0.0001.}
\label{figure:df-hypothesis-test-formatted}
\end{figure}
% This latex figure presents "df_hypothesis_test_formatted.png",
% which was created from the df:
%
% index,"Coefficient","CI Low","CI High","P-value"
% "Query Length: Zookeeper",-0.02433,-0.04558,-0.003084,0.0249
% "Query Length: Proxifier",-0.02748,-0.05749,0.002525,0.0725
% "Query Length: BGL",-0.0211,-0.04655,0.004338,0.104
% "Query Length: HDFS",-0.0169,-0.03808,0.004285,0.118
% "Query Length: Mac",-0.01667,-0.03758,0.00424,0.118
% "Query Length: Spark",-0.01706,-0.03901,0.004878,0.127
% "Query Length: Hadoop",-0.00917,-0.02955,0.01121,0.377
% "Query Length: Thunderbird",-0.008408,-0.02979,0.01297,0.44
% "Query Length: OpenSSH",0.008924,-0.01647,0.03432,0.49
% "Query Length: HealthApp",0.009192,-0.01999,0.03837,0.536
%
% To create the figure, this df was plotted with the command:
%
% df.plot(x=None, y=['Coefficient'], xlabel='System Type', ylabel
    ='Interaction Coefficient', kind='bar')
%
% Confidence intervals for y-values were then plotted based on
    column: [('CI Low', 'CI High')].
%
% P-values for y-values were taken from column: ['P-value'].
%
% These p-values were presented above the data points as stars (
    with significance threshold values indicated in the figure
    caption).
```